Recommender Systems Ultimate Cheat Sheet

End-to-End Guide: From Matrix Factorization to Deep Learning

1. INTRODUCTION TO RECSYS

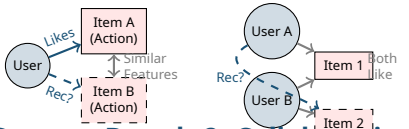
Definition: Algorithms that suggest relevant items to users.

Concepts

- **Personalization:** Tailoring results to individual user history (e.g., Netflix).
- **Relevance:** How useful the item is to the user now.
- **Ranking vs Search:** Search is active (query-based); Recommendation is passive (context-based).

2. TYPES OF SYSTEMS

Approaches



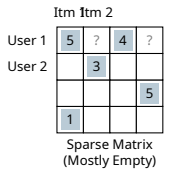
1. Content-Based **2. Collaborative**

- **Content-Based:** Recommends items similar to what user liked before (based on metadata).
- **Collaborative Filtering (CF):** Recommends items liked by similar users.
- **Hybrid:** Combines both (e.g., weighted average).
- **Knowledge-Based:** Uses explicit rules (e.g., "I want a laptop under $1000").

3. DATA FOR RECSYS

User-Item Matrix

Core data structure. Rows = Users, Cols = Items.



Sparse Matrix (Mostly Empty)

Feedback Types

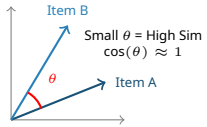
- **Explicit:** Ratings, Likes, Thumbs up. High signal, low volume.
- **Implicit:** Clicks, Purchase, Watch time. Low signal, high volume.

Challenges

- **Cold Start:** New user/item has no history.
- **Sparsity:** Matrix is 99% empty. Hard to find neighbors.

4. MATHEMATICAL INTUITION

Similarity Measures



Key Concepts

- **Similarity:** Cosine, Jaccard, Pearson.
- **Dot Product:** $u \cdot v = \sum u_i v_i$. Fundamental score calculation.
- **Latent Factors:** Hidden features (e.g., "Genre", "Style") learned by matrix factorization.
- **Loss Function:** MSE (Rating prediction) or LogLoss (Click prediction).

5. CONTENT-BASED FILTERING

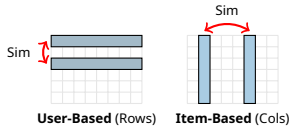
Process

- **Features:** TF-IDF for text, Genres for movies.
- **Profile:** Aggregated vector of items user liked.
- **Prediction:** Sim$(User_{profile}, Item_{features})$.

```
from sklearn.metrics.pairwise import cosine_similarity
# Calculate similarity between items
sim_matrix = cosine_similarity(tfidf_matrix)
```

6. COLLABORATIVE FILTERING

Memory-Based Methods

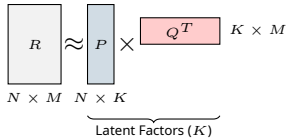


- **User-User CF:** Find users like Alice. Recommend what they liked.
- **Item-Item CF:** Find items usually bought together. (Amazon style: "Users who bought this...").
- **Pros:** Explainable. **Cons:** Scalability issues.

7. MATRIX FACTORIZATION

Decomposition

Factorize User-Item matrix (R) into low-rank matrices (U, V).



Algorithms

- **SVD (Singular Value Decomp):** Mathematical baseline.
- **ALS (Alternating Least Squares):** Parallelizable, handles implicit data well.
- **SGD:** Stochastic Gradient Descent optimization.

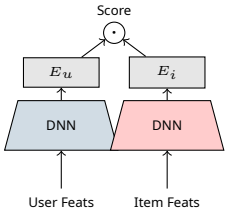
8. DEEP LEARNING MODELS

Neural Collaborative Filtering (NCF)

Replaces dot product with Neural Network (MLP) to learn non-linear interactions.

Two-Tower Model

Industry Standard (YouTube, Spotify).



Advanced Architectures

- **Sequence Aware:** RNN/Transformer (SASRec) for user history order.
- **Autoencoders:** Denoising inputs to predict missing ratings (AutoRec).

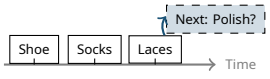
9. CONTEXT & SESSION BASED

Context

- **Time:** Morning vs. Night preferences.
- **Location:** Recommend nearby restaurants.

Session Based (Sequence)

For anonymous users. Uses sequence of recent clicks.



Sequence Modeling (RNN/Transformer)

10. IMPLEMENTATION (PYTHON)

Surprise (Explicit Feedback)

```
from surprise import SVD, Dataset
# Load data and fit SVD
data = Dataset.load_builtin('ml-100k')
algo = SVD()
algo.fit(data.build_full_trainset())
pred = algo.predict(uid='196', iid='302')
```

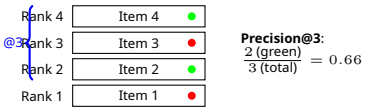
Neural Network (PyTorch)

```
class NCF(nn.Module):
    def __init__(self, n_users, n_items, embed_dim):
        super().__init__()
        self.u_emb = nn.Embedding(n_users, embed_dim)
        self.i_emb = nn.Embedding(n_items, embed_dim)
        self.fc = nn.Linear(embed_dim * 2, 1)

    def forward(self, u, i):
        x = torch.cat([self.u_emb(u), self.i_emb(i)], dim
        =1)
        return torch.sigmoid(self.fc(x))
```

11. PERFORMANCE METRICS

Ranking Metrics (Precision@K)



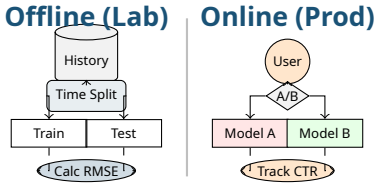
- **Precision@K:** % of top-K items that are relevant.
- **Recall@K:** % of all relevant items found in top-K.
- **MAP:** Mean Average Precision. Accounts for rank order.
- **NDCG:** Normalized Discounted Cumulative Gain. Rewards highly relevant items at top ranks.

Non-Accuracy Metrics

- **Coverage:** % of items catalog recommended.
- **Diversity:** How different recommended items are.
- **Novelty:** Recommending unpopular/unseen items (Serendipity).

12. EVALUATION STRATEGY

Offline vs Online



- **Time-Split:** Train on past, test on future (Avoid data leakage).
- **A/B Testing:** Real-user test. Group A gets Algo 1, B gets Algo 2.
- **Interleaving:** Mix results from A and B, see which gets clicked.

13. PRACTICAL CHALLENGES

- **Cold Start:** Use Popularity baseline or Content-based for new users.
- **Bias:** Popularity bias (rich get richer).
- **Scalability:** Approximate Nearest Neighbors (FAISS, Annoy) for serving.
- **Feedback Loops:** Model trains on its own biased outputs.

14. REAL WORLD USE CASES

- **E-commerce:** "Frequently bought together" (Association rules/Item2Vec).
- **Streaming:** Next song/video (Sequence models).
- **Ads:** CTR prediction (Wide & Deep).

15. SUMMARY MATRIX

Content-Based: Best for new items, high text data. No community data needed.

Collaborative: Best for discovery, high serendipity. Needs interaction history.

Matrix Fact: Standard baseline. Good for static data.

Deep Learning: Best for massive scale, mixed features (images/-text), and complex contexts.